

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)**

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

«Курсовая работа
по дисциплине: «База данных»
на тему: «Библиотека на базе телеграмм бота»

Автор работы _____ Дорошенко Владимир Юрьевич
(Подпись)

Руководитель работы _____ Панченко Максим Владимирович
(Подпись)

Оценка _____

Белгород, 2024

Оглавление

Введение.....	1
Постановка задач.....	2
Выполнение	3
ER-модель	3
Описание функционала	6
Демонстрация	8
Заключение	13
Github.....	13
Список литературы	14

Введение

В современном мире информационные технологии стали одной из основ для развития общества. Базы данных, как один из ключевых инструментов, обеспечивают хранение и обработку огромных объёмов данных в различных областях деятельности.

Целью данной курсовой работы стало создание приложения, которое упрощает взаимодействие с базой данных. Основное внимание уделено функциональности, которая отвечает современным требованиям: актуализация данных, гибкость настроек и поддержка популярных форматов экспорта. Важной особенностью разработки является интеграция приложения с Telegram, что делает его доступным и интуитивно понятным для пользователей.

Разработка подобного приложения помогает пользователям эффективно управлять информацией, в том числе анализировать и получать своевременные уведомления. В ходе работы были изучены технологии взаимодействия с базой данных, интеграция приложений с API внешних сервисов и автоматизация резервного копирования.

Постановка задач

Для успешной реализации проекта были сформулированы следующие задачи:

1. Разработать архитектуру базы данных, включающую несколько таблиц для хранения информации о пользователях, оповещениях и криптовалютах.
2. Реализовать удобный и интуитивно понятный пользовательский интерфейс.
3. Обеспечить возможность настройки оповещений для пользователей.
4. Реализовать функции выгрузки данных из базы данных в форматы JSON, CSV и XLSX.
5. Настроить механизм автоматического резервного копирования базы данных и её загрузки на удалённый сервер или в облачное хранилище.

Выполнение

Реализация проекта включала несколько ключевых этапов:

1. Проектирование базы данных

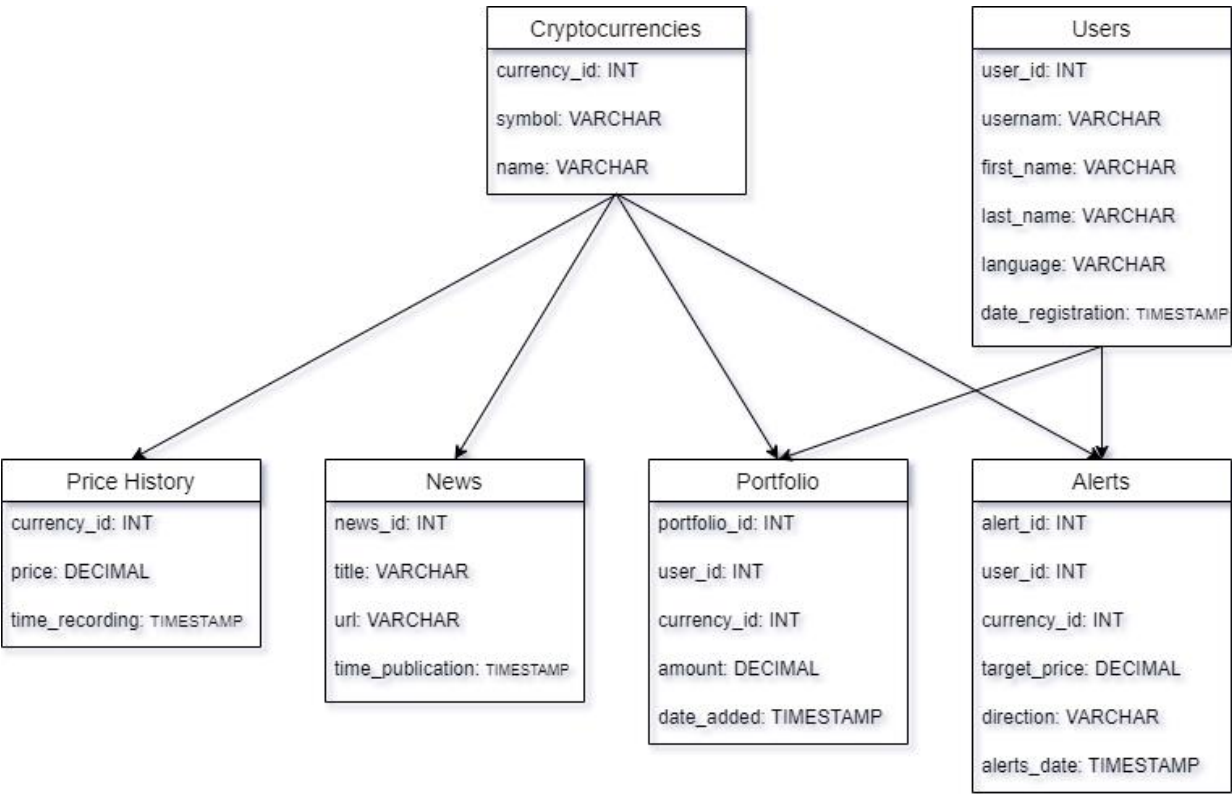
Было создано логическое и физическое представление базы данных, включающее три основные таблицы:

users: хранение информации о пользователях, включая их уникальные идентификаторы и настройки.

alerts: управление пользовательскими уведомлениями, включая их содержание и частоту.

cryptocurrencies: таблица для хранения данных о криптовалютах, таких как символ, название, текущий курс и дата обновления.

Схема была представлена в виде ER-диаграммы, что обеспечило наглядное понимание связей между таблицами.



1. Таблица пользователей (users)

Хранит информацию о пользователях бота.

Описание таблицы users

Название столбца	Тип данных	Описание	Обязательный
id	serial	Уникальный идентификатор записи	Да
id_user	varchar(50)	Идентификатор пользователя	Да
first_name	varchar(50)	Имя пользователя	Нет
last_name	varchar(50)	Фамилия пользователя	Нет
language	varchar(10)	Язык пользователя	Да
chat_id	integer	Уникальный идентификатор чата Telegram	Да
last_alert_id	bigint	Идентификатор последнего оповещения	Нет

2. Таблица валют (cryptocurrencies)

Хранит информацию о криптовалютах, которые могут отслеживать пользователи.

Описание таблицы cryptocurrencies

Название столбца	Тип данных	Описание	Обязательный
currency_id	serial	Уникальный идентификатор записи	Да
symbol	varchar(50)	Символ криптовалюты (например, "BTC", "ETH")	Да
name	varchar(50)	Название криптовалюты	Да
market_cap	numeric(20, 2)	Рыночная капитализация (в долларах)	Нет
price	numeric(20, 10)	Текущая цена криптовалюты	Нет
price_change_percentage_24h	numeric(20, 2)	Изменение цены за 24 часа (в процентах)	Нет
volume_24h	numeric(20, 2)	Объём торгов за последние 24 часа (в долларах)	Нет
last_updated	timestamp	Дата и время последнего обновления данных	Нет

3. Таблица оповещений (alerts)

Хранит настройки оповещений для каждого пользователя.

Описание таблицы alerts

Название столбца	Тип данных	Описание	Обязательный
alert_id	serial	Уникальный идентификатор уведомления	Да
user_id	integer	Ссылка на ID пользователя из таблицы <code>users</code>	Да
symbol	varchar(50)	Символ криптовалюты, для которой создаётся оповещение	Нет
target_price	integer	Целевая цена криптовалюты	Нет
direction	varchar(10)	Направление изменения цены ("up" или "down")	Да
alerts_date	timestamp	Дата и время создания уведомления	Нет
number_repetitions	integer	Количество повторений уведомления	Нет

2. Создание Telegram-бота

Telegram-бот выступает пользовательским интерфейсом приложения. С его помощью реализованы:

Регистрация и авторизация пользователей.

Отображение данных о криптовалютах.

Возможность настроить уведомления об изменениях курсов.

Управление частотой уведомлений через интерактивные кнопки.

3. Добавление ORM технологии

```
4. from sqlalchemy import create_engine, Column, Integer, String, Numeric,
    ForeignKey, BigInteger, DateTime, Sequence
    from sqlalchemy.ext.declarative import declarative_base
    from sqlalchemy.orm import relationship
    from sqlalchemy.orm import sessionmaker

    Base = declarative_base()

    DATABASE_URL = f"postgresql://{user}:{password}@{host}/{db_name}"
    engine = create_engine(DATABASE_URL, echo=False)
    Session = sessionmaker(bind=engine)
    session = Session()

    Base.metadata.create_all(engine)

    class Alerts(Base):
        __tablename__ = 'alerts'

        alert_id = Column(Integer, Sequence('alerts_alert_id_seq'),
            primary_key=True)
        user_id = Column(Integer, ForeignKey('users.id',
            ondelete='CASCADE'))
        symbol = Column(String(50))
        target_price = Column(Numeric(20, 10))
        direction = Column(String(10), nullable=False)
        alerts_date = Column(DateTime)
        number_repetitions = Column(Integer, default=1)

        user = relationship("Users", back_populates="alerts")

    class Cryptocurrencies(Base):
        __tablename__ = 'cryptocurrencies'

        currency_id = Column(Integer,
            Sequence('cryptocurrencies_currency_id_seq'), primary_key=True)
        symbol = Column(String(50), unique=True, nullable=False)
        name = Column(String(50), nullable=False)
        market_cap = Column(Numeric(20, 2))
        price = Column(Numeric(20, 10))
        price_change_percentage_24h = Column(Numeric(20, 2))
        volume_24h = Column(Numeric(20, 2))
        last_updated = Column(DateTime)
```

```
class Users(Base):
    __tablename__ = 'users'

    id = Column(Integer, Sequence('users_id_seq'), primary_key=True)
    id_user = Column(String(50), nullable=False)
    first_name = Column(String(50))
    last_name = Column(String(50))
    language = Column(String(10), nullable=False)
    chat_id = Column(Integer, nullable=False)
    last_alert_id = Column(BigInteger)

    alerts = relationship("Alerts", back_populates="user")
```

5. Интеграция с внешними API

Использовались API сервисов, предоставляющих данные о криптовалютах. Это позволило получать актуальные курсы в режиме реального времени и обновлять базу данных каждые несколько минут.

6. Экспорт данных

Был реализован механизм выгрузки данных из базы в три популярных формата: JSON, CSV и XLSX. Для этого использовались стандартные библиотеки Python (json, csv) и библиотека openpyxl для работы с Excel. Это позволяет пользователям извлекать данные в удобном для анализа виде.

7. Резервное копирование

База данных регулярно сохраняется в виде резервных копий, которые автоматически загружаются в облачное хранилище Google Cloud Storage. Для этого был использован SDK Google Cloud, обеспечивающий безопасную передачу данных.

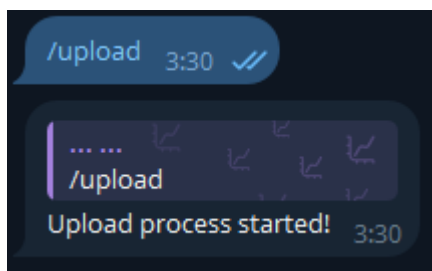
Эти этапы были выполнены с учётом гибкости системы, что позволяет её дальнейшее развитие и масштабирование, например, добавление новых таблиц или интеграций с другими сервисами.

Описание функционала

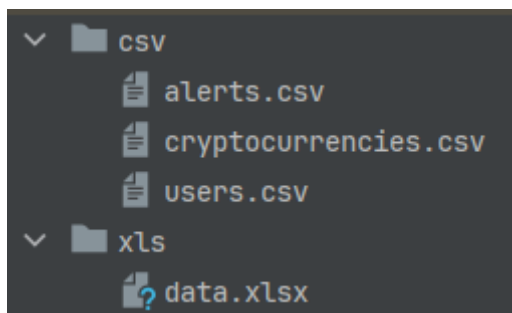
Приложение включает следующие основные функции:

1. **Регистрация пользователей:** Пользователи могут зарегистрироваться через Telegram-бот, после чего их данные сохраняются в базе.
2. **Настройка уведомлений:** Пользователи могут создавать оповещения, задавать условия для их срабатывания и редактировать параметры. Для удобства реализованы кнопки управления.
3. **Просмотр текущей цены криптовалюты:** Пользователи могут запросить текущую стоимость интересующей криптовалюты через Telegram-бот. Информация обновляется в режиме реального времени.
4. **Выгрузка данных:** Поддерживается экспорт данных из базы в форматы JSON, CSV и XLSX. Например, пользователи могут выгрузить список своих оповещений или информацию о криптовалютах.

Команда для выгрузки всех БД



Выгрузка происходит в папку с проектом



5. **Автоматическое обновление данных:** Курс криптовалют обновляется каждые несколько минут с использованием внешних API, а данные записываются в таблицу cryptocurrencies.

6. **Резервное копирование базы данных:** Реализована автоматическая выгрузка резервных копий базы данных на Google Cloud Storage.

Написал скрипт, который выгружает backup в Dropbox

```
import os
import schedule
import time
from datetime import datetime
import dropbox

# Конфигурация
DB_NAME = "crypto_bot"
DB_USER = "postgres"
DB_HOST = "127.0.0.1"
DB_PORT = "5432"
BACKUP_DIR = "C:\\Backups\\crypto_bot"
DROPBOX_ACCESS_TOKEN =
"s1.u.AFZASxgHsImWU0Tu66eQF3BtUc3jiU8ATV7ViAQaZDJlBoMxcBQSuu8hWbox-
i3OwTgerA4C7u6UovYP_MhEnMR7zke4WTi3TJoqpmDzGzS2bRnFN9Jz9eE9NesN5ETcFQBy5tTXE9
OBTxojnUINZDC1Ex5x2D6FjTlFjfs8eIUEaMDnjjiFBs1VF8dokyC0yTiWManmYglN0reUXysMnp8
KEbe6b58440ekxAnC7EqwaWFe4rR5IVn1TVmEu3Wzu-
uUkLqFqUFdJga0QF1LyY3G1hGaxqka51_O3fnSJSCCrPRmUw2DSc9QmcEgulDYtMmTICm744admj
9c0clcYEjNan1zVrzAsnxakK0kpqxNmc7yKYtVYCYeAPZejVXMYc19-YDceFz3cC4aqKfg_UC-
qVmvhgd3gam2MUQ-OGpxqMEpqtoYx2qPDk7d6ywUZvc7jaMYM84XHfOJJ_zfbbFG8NHPV5kRUrEE-
va_zFPY6w1wWYgRUHuVZZCoKl4Lk-tTpg8-1fZqTmlTiUD7Cgcm1ICgLk75m8-aeH-
U4EozfGVToS9sukClgqOxCBjGGNpeeSHCMGMzR6Jo_glJelgv80Y2j12pUKz69IfIp57J6tYSCot6
jFcqHu4Qh1ZVVHQU69KrlY_g8Haenp1N1ZNcBdY2NXiIX0dPmQUCMLBUHJW_wUIWwGh0w_f2242cp
XpjybcruFckZeXI6puIUlLgsp9mxIguXbb87RLFg3c8EhCG8elpdRbaXfPvcIi6YjTb6QHKRhw7Tw
_nipLeLxP6e7vCNmva3fnhUw7J9SfWnvtjvteghpEZSiHbRoNaFpMvJaUwJQCJJam2SyP4KXcJrrF
W2yhRgyE7ak8lSDQ85_tUfUtAZfyySkaWkBhgScvLPG2_sdkH2pJlG5pTk-3KPkkdHXVYSPjEE-
Ywa9QoaBa26vD-f8FCCh3hrAOexrY39SL6-hGYdjigJY25g7ux6B-
Xba8B5_Nzzkub7Yh_kD6z5hDGJaCWzwlqix2Uv5kgOgsqX0Ktkz1Ccms9Mm7DlPL3z2ELnrDFg4LS
-5lIWtiy4cw0zWEK3fcZ1p77IWHpQkKI39PB1K8xPqTrfarWaf--L-
fJcgBm5YbBma7ZQzfsR_9iqME-v8OZDxtxQZOqch_BnasZmD5-
Pa785Wgm851qCbH8uXuTBxIZ1fyj_OBOaxo_lj6A3DgdGkXbVLL4DJ6g-ESrjxjeWxNI9R9-
GPbSsPRyKUFQCv0zg_z2u8XzIMDM3jEEF1bfo_aNldzR9_oxUV3HE1NHMCgIW6Tk91x0vNgK1BBk
rBVGMrkHC_AW6bRsbPTmwzpZIC9Rnc8PsVNngnWPWLprZJF1VK8tPvbjpg4tya3o5ltsFLD8NeHyZq
CNFIGGxyBUFSdKok21f1FxqZ9-x6k8UEHayWNrUglUdHSjleUDcG0JwmyzO-
VLVCxDOIyelCpCze3G2Psk9JkYxnR1yX1v3V2Havd8l5jHdFeW9YEKiFnvQK0A"
DROPBOX_FOLDER = "/backups"

# Создание резервной копии
def create_backup():
    try:
        if not os.path.exists(BACKUP_DIR):
            os.makedirs(BACKUP_DIR)

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        backup_file = os.path.join(BACKUP_DIR, f"backup_{timestamp}.sql")

        command = [
            "pg_dump",
```

```

        f"-U{DB_USER}",
        f"-h{DB_HOST}",
        f"-p{DB_PORT}",
        DB_NAME,
        f"> {backup_file}"
    ]

    # Выполнение команды через shell
    os.system(" ".join(command))

    if os.path.exists(backup_file):
        print(f"Резервная копия создана: {backup_file}")
        upload_to_dropbox(backup_file)
    else:
        print("Ошибка: Резервная копия не была создана.")

except Exception as e:
    print(f"Ошибка при создании резервной копии: {e}")

def upload_to_dropbox(local_file_path):
    try:
        dbx = dropbox.Dropbox(DROPBOX_ACCESS_TOKEN)

        with open(local_file_path, "rb") as f:
            file_name = os.path.basename(local_file_path)
            dropbox_path = f"{DROPBOX_FOLDER}/{file_name}"

            dbx.files_upload(f.read(), dropbox_path,
mode=dropbox.files.WriteMode.overwrite)

            print(f"Файл успешно загружен в Dropbox: {dropbox_path}")

    except Exception as e:
        print(f"Ошибка при загрузке в Dropbox: {e}")

# Планирование задачи
schedule.every().day.at("02:00").do(create_backup)

print("Автоматический бэкап настроен. Скрипт запущен...")
while True:
    schedule.run_pending()
    time.sleep(1)

```

Демонстрация

Старт

/start 23:46 ✓✓

A stylized illustration on a dark blue background. In the center is a white outline of a man in a suit and tie, holding a clipboard with a Bitcoin logo and a checkmark. Surrounding him are various crypto-related icons: a large Bitcoin symbol, a candlestick chart, a bar chart, and a network diagram.

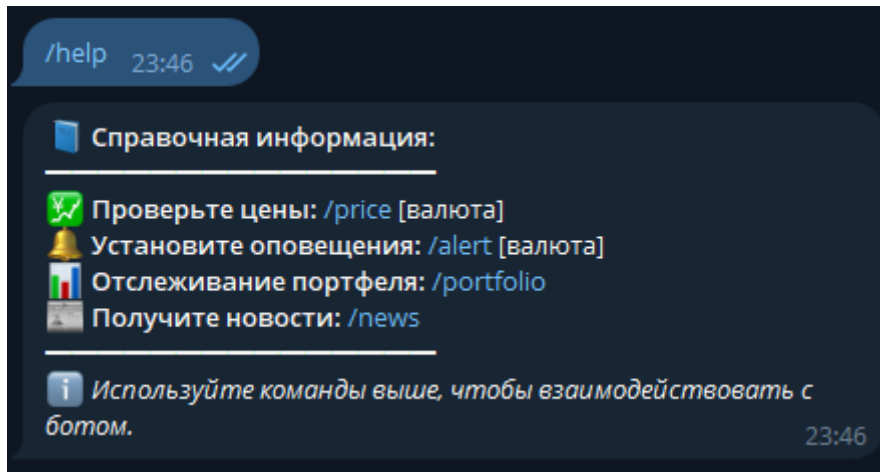
... .. Добро пожаловать в CryptoTrackerBot! 📈🚀
Ваш персональный помощник для отслеживания и анализа
рынка криптовалют.
Вот что вы можете делать:

- 💰 Проверка цен: используйте `/price` [валюта] (например, `/price BTC`), чтобы узнать последние рыночные цены.
- 🔔 Установка оповещений: используйте `/alert` [валюта] [цена] (например, `/alert ETH 2000`), чтобы установить пользовательские оповещения о ценах.
- 📊 Отслеживание портфеля: управляйте своими активами и просматривайте обновления в реальном времени с помощью `/portfolio`.
- 📰 Получение новостей: будьте в курсе последних новостей, введя `/news`.

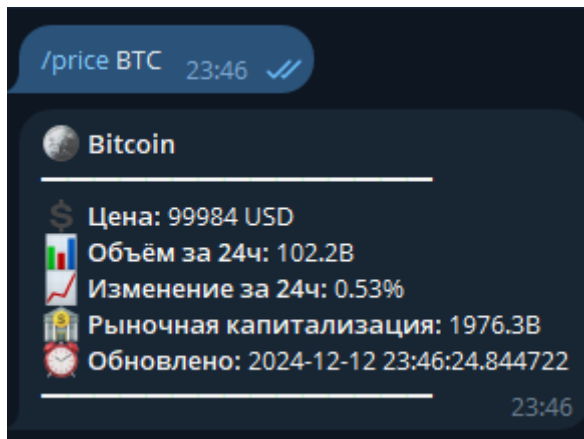
Введите `/help`, чтобы увидеть все доступные команды.
Давайте следить за рынком вместе! 🇺🇸🔥

23:46

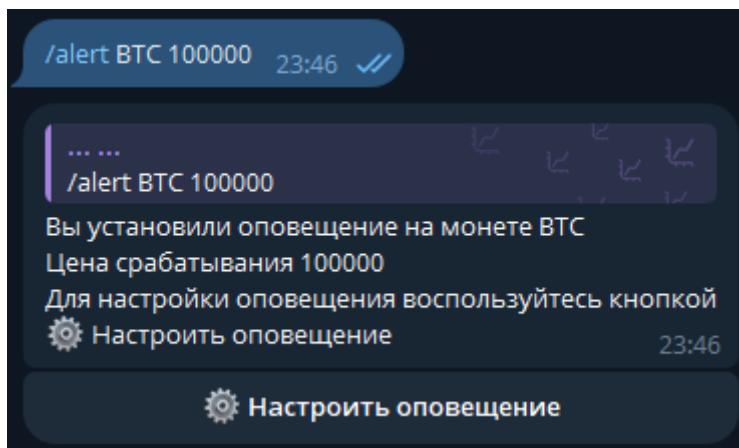
Информация на старте для понимания работы бота



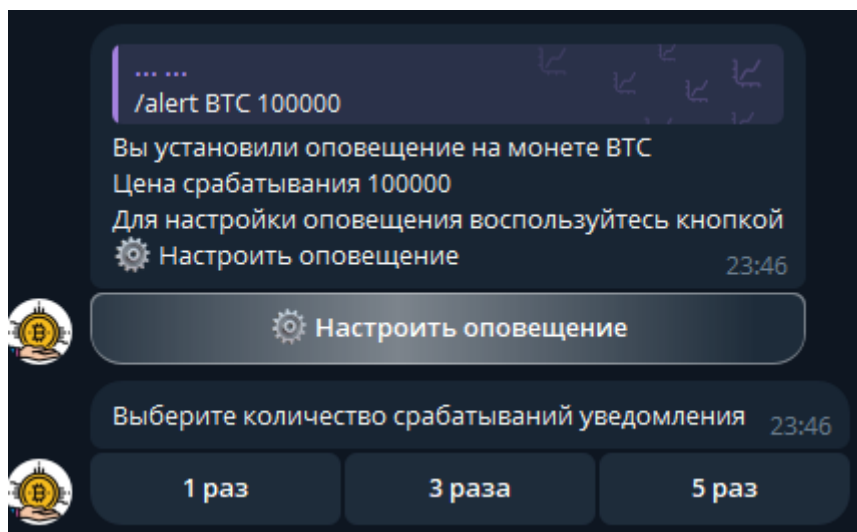
Просмотр цены криптовалюты в реальном времени



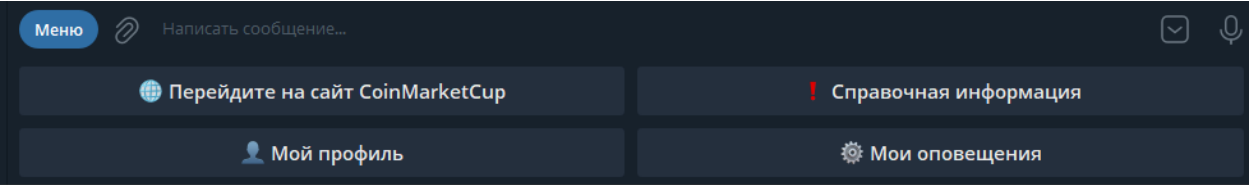
Установка оповещения достижения цены



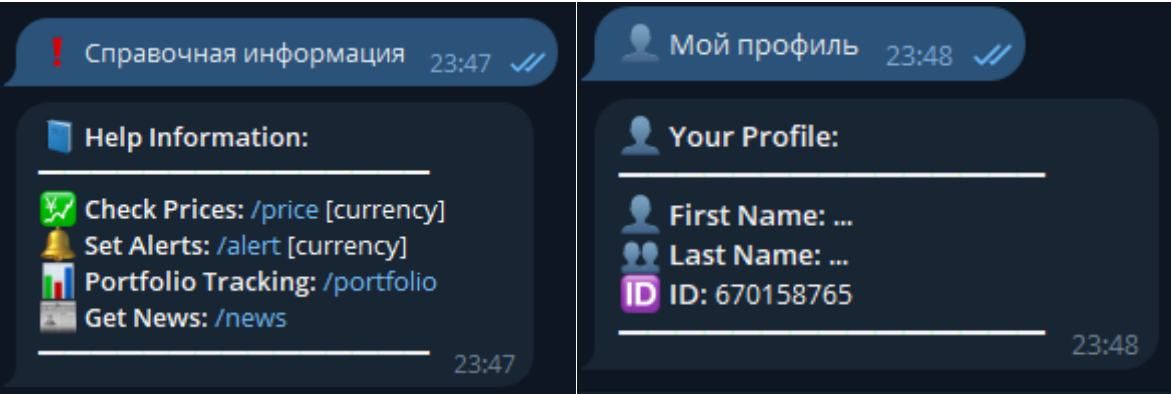
Настройка установленных оповещений



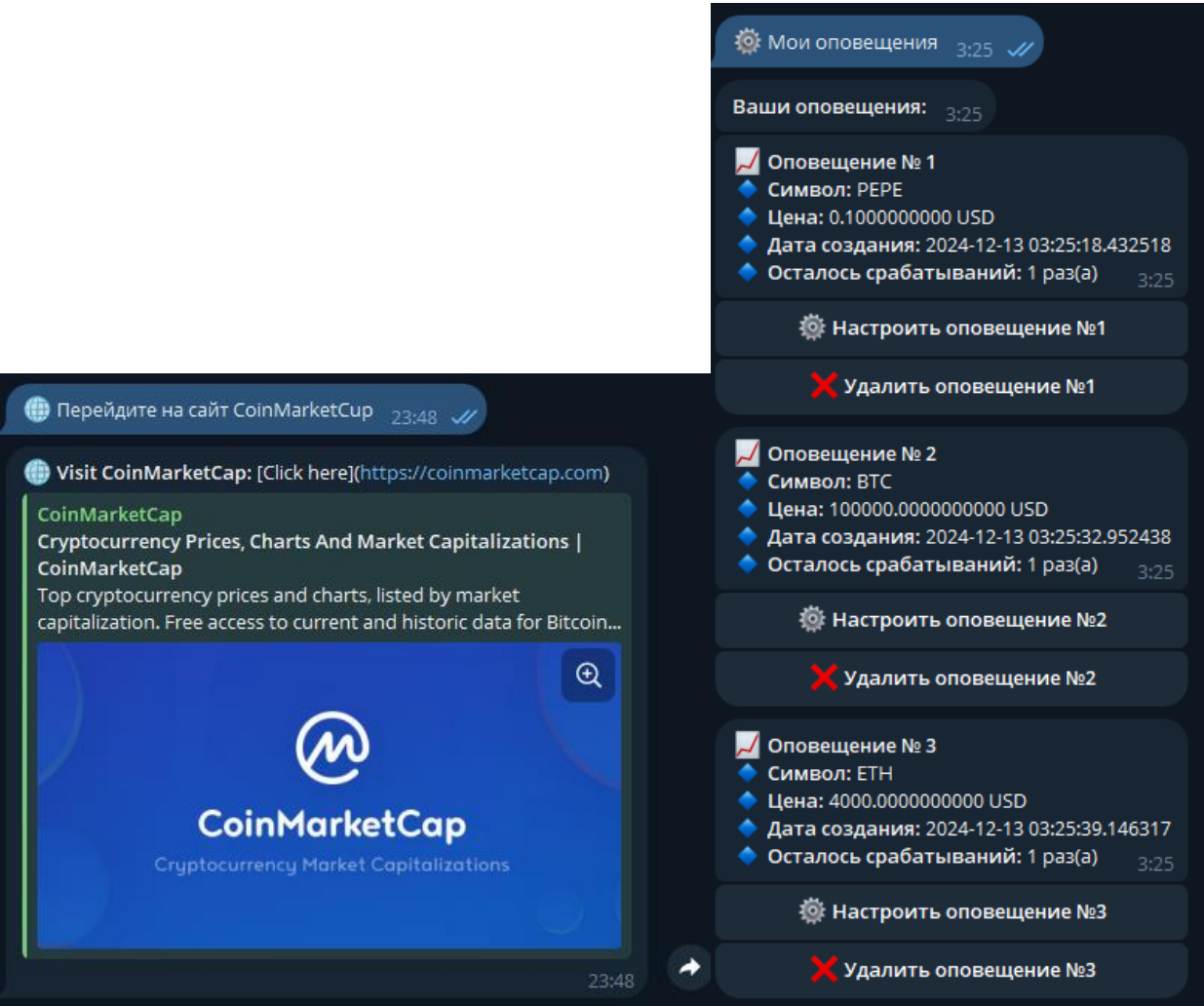
Меню кнопок на панели



Справочная информация и информация о профиле



Сайт откуда беру данные и просмотр всех моих уведомлений с возможностью настройки



Заключение

Курсовая работа продемонстрировала возможности эффективной интеграции базы данных с внешним интерфейсом и автоматизацией процессов.

Полученное приложение предоставляет пользователям удобный инструмент для работы с данными и может быть легко масштабировано для расширения функционала.

Github project

https://github.com/wwwxtus/telegram_crypto_bot.git

Ссылка на бота в телеграмме

@crypto_cointracker_bot

Список литературы

1. Книга: С. Рей, «PostgreSQL: Руководство пользователя».
2. Документация по библиотеке pyTelegramBotAPI:
<https://github.com/eternnoir/pyTelegramBotAPI>
3. Официальная документация PostgreSQL: <https://www.postgresql.org/docs/>
4. API для получения данных о криптовалютах:
<https://www.coingecko.com/en/api>
5. Руководство по Google Cloud Storage: <https://cloud.google.com/storage/docs>